

**AHSANULLAH UNIVERSITY OF SCIENCE AND  
TECHNOLOGY (AUST)**

141 & 142, Love Road, Tejgaon Industrial Area, Dhaka-1208.

Department of Computer Science and Engineering  
Program: Bachelor of Science in Computer Science and Engineering

**Project Final Report**

Course No: CSE-1200

Course Title: Software Development – I

Section: C2

Project Title: Honda Rush: Career Rider

Date of Submission: 30/09/26

Submitted to

Saha Reno

Assistant Professor, Department of CSE, AUST.

Mustofa Ahmed

Lecturer, Department of CSE, AUST.

Submitted by,

Name: Mujahidul Islam Ansary Niloy

Student ID: 00725105101164

Name: Foysal Sami

Student ID: 00725105101149

Name: Parijat Sarker

Student ID: 00725105101152

# 1. Introduction

## 1.1 Project Overview

Honda Rush: Career Rider is a 2D side-scrolling motorcycle game written in C++ with the iGraphics library. The player rides a Honda through three levels, each laid out as a fixed route of full-screen background panels with a playable road drawn in front of them. Riding means managing speed, fuel and timing: ramps have to be taken fast enough to clear the obstacle waiting on the other side, coins are collected along the way, and fuel cans keep the tank alive.

The three levels are deliberately different in character. Level 1 is an eight-screen introduction that teaches the controls, where an early collision only costs speed and a later one ends the run. Level 2 is a ten-screen stunt route built around five ramps and five obstacles. Level 3 is the longest: thirteen backgrounds, robbers who have to be fought on foot, cars and tanks on the road, a 360-degree loop, and a story in which the rider collects a passenger and takes her to AUST. The game runs on Windows in a fixed  $1280 \times 720$  window.

## 1.2 Objective

- To develop a complete 2D motorcycle game in C++ using the iGraphics library.
- To implement bike physics — acceleration, braking, jumping, gravity and landing.
- To build ramps whose riding surface matches the artwork instead of an invisible shape.
- To create three levels with fixed layouts, so every run of a level is identical.
- To implement collision detection against obstacles, vehicles and enemies.
- To develop coin, score and fuel systems with an on-screen HUD.
- To add an on-foot mode with shooting, so the player can leave the bike and fight.
- To implement a menu, level selection, dialogue subtitles, cheat codes, game over and level complete screens.
- To play background music and sound effects through the Windows Multimedia API.

# 2. Game Development

## 2.1 Tools & Technologies Used

- **Programming language:** C++
- **IDE:** Visual Studio 2013 (v120 toolset, Win32)
- **Graphics library:** iGraphics, over OpenGL and GLUT
- **Image loading:** stb\_image.h — PNG and JPEG artwork
- **Audio:** Windows Multimedia API (winmm / MCI)
- **Operating system:** Windows
- **AI assistance:** Claude

## 2.2 Core Features

### *Bike Movement*

D accelerates, A brakes and reverses, W jumps and S pushes the bike down faster while it is in the air. Speed decides which riding picture is drawn, so the bike visibly changes posture as it gets faster. The bike is held at a fixed column on the screen and the world scrolls past it.

### ***Ramps and Jumping***

A ramp is not a decorative picture with a triangle behind it. When a ramp loads, the height of its top edge is measured column by column from the alpha channel of its own artwork, giving a surface profile that matches the drawing exactly. The wheels are then placed on that profile, so the bike climbs the shape the player can see. Leaving the crest converts the climb rate into upward velocity, which means a faster approach produces a higher jump. The launch is capped so that the bike never leaves the top of the screen.

### ***Road Contact***

The bike rests on two measured contact points, one under each wheel, and sits on whichever is higher. On a slope it is tilted to match, so both tyres stay on the surface instead of one sinking in and the other hanging in the air.

### ***Coins, Score and Fuel***

Coins are worth 5, 10, 30, 50 and 100 and are placed at fixed world positions, including arcs over the obstacles that can only be collected in mid-air. Fuel drains while the bike is moving and is topped up by fuel cans along the route; running out ends the level.

### ***On-foot Mode (Level 3)***

Pressing ENTER when the bike has stopped for a fight puts the rider on the road. Walking, running, jumping and shooting have their own handling, completely separate from the bike physics, so the two control sets can never interfere with each other. The rider gets back on the same way once the fight is cleared.

### ***Enemies and Vehicles***

Level 3 has two types of robber with their own walking, attacking and death animations, and each fight has its own limit on how many hits the player can take. Cars drive towards the bike and have to be jumped, and tanks roll up and hold the road until they are shot or jumped.

### ***Cheat Codes***

Four codes are typed during play. MACHINEGUNON gives the rider a gun, GETONTHEBIKE brings the girl onto the bike in BG6, GIVEPERMISSION clears the army checkpoint, and COMPLETEYOURPAYMENTQUICKLY pays the semester fee within a twenty-second limit.

### ***Audio***

Each level has its own background music, opened once through MCI and played under an alias. The track keeps playing as the backgrounds change and only stops on a win or a game over. Collision, coin and result sounds are played as separate short effects on top of the music.

## 3. Implementation Details

### 3.1 Code Structure

The project is built as a single translation unit. `iMain.cpp` is the only `.cpp` file that is compiled, and every other part of the game is a header that `iMain.cpp` includes in a fixed order. The headers are therefore a way of organising the code rather than separately compiled modules. `iGraphics.h` in particular has no include guard, so it is included exactly once, before everything that depends on it.

| File                         | Responsibility                                                                                                       |
|------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>iMain.cpp</code>       | Starts the game, owns the game loop, and routes every draw, mouse and keyboard callback to whichever screen is open. |
| <code>iGraphics.h</code>     | The graphics library: window, drawing primitives, timers and input.                                                  |
| <code>stb_image.h</code>     | Loads the PNG and JPEG artwork.                                                                                      |
| <code>Config.h</code>        | Window size, game states, colours, and the shared image and sprite-sheet loaders.                                    |
| <code>Input.hpp</code>       | Edge-triggered key helpers, so a tap can be told apart from a held key.                                              |
| <code>Audio.hpp</code>       | Background music and sound effects through MCI.                                                                      |
| <code>Button.hpp</code>      | The menu button widget.                                                                                              |
| <code>MenuScreen.hpp</code>  | Intro poster and the level selection menu.                                                                           |
| <code>KeysScreen.hpp</code>  | The keyboard control list.                                                                                           |
| <code>AboutScreen.hpp</code> | The project summary screen.                                                                                          |
| <code>Level01.hpp</code>     | Level 1: eight backgrounds, two ramps, four obstacles, coins and fuel.                                               |
| <code>Level02.hpp</code>     | Level 2: ten backgrounds, five ramps and five obstacles, coins and fuel.                                             |
| <code>Level03.hpp</code>     | Level 3: thirteen backgrounds, the fights, vehicles, the loop, the story events and the cheats.                      |

### 3.2 How a Level is Built

All three levels share the same idea. The backgrounds are laid end to end and the camera slides along them, so the order of the panels is a property of the layout rather than something a timer decides — they cannot be shuffled or skipped. Every ramp, obstacle, coin and fuel can is given a fixed world position when the level is built, which means a level plays exactly the same way every time it is started. Objects are drawn at their world position minus the camera, so they stay in step with the scrolling world automatically.

In Level 3 the world turns around halfway through. After the girl gets on the bike in BG6, the remaining panels are laid out to the left of BG6 instead of the right, and the bike rides right to

left through BG7 to BG13. The pictures themselves are never mirrored; only their positions change.

## 4. Screenshots



Figure 1: The main menu. The intro poster is on the left and the level selection on the right — LEVEL 1, LEVEL 2, LEVEL 3, GAME KEY and EXIT. The intro music plays behind this screen and only stops when a level starts.

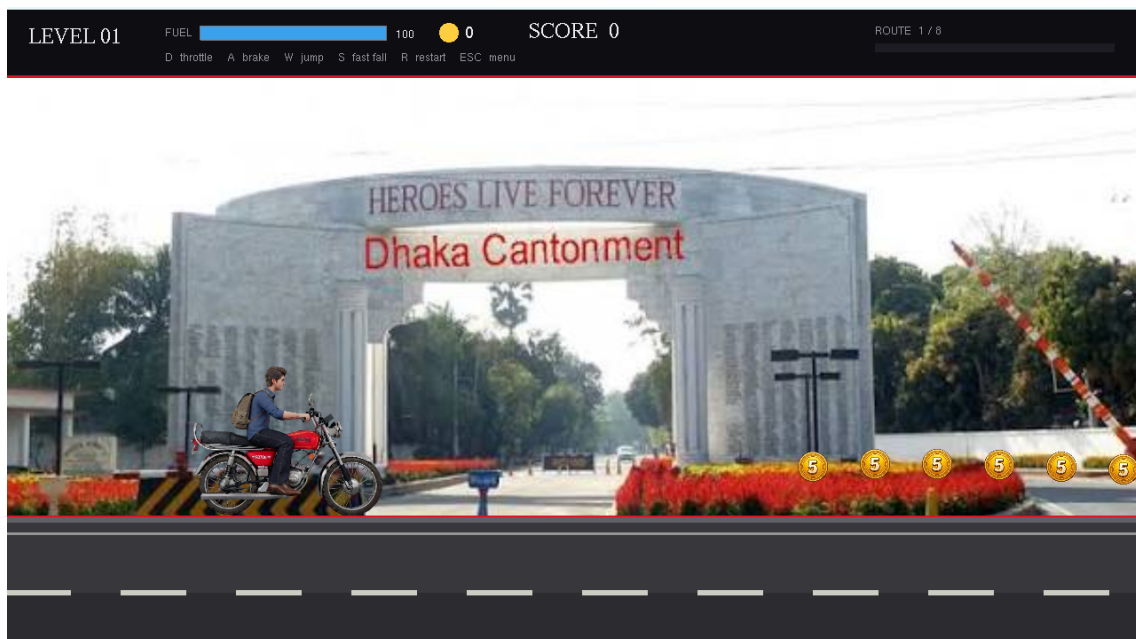
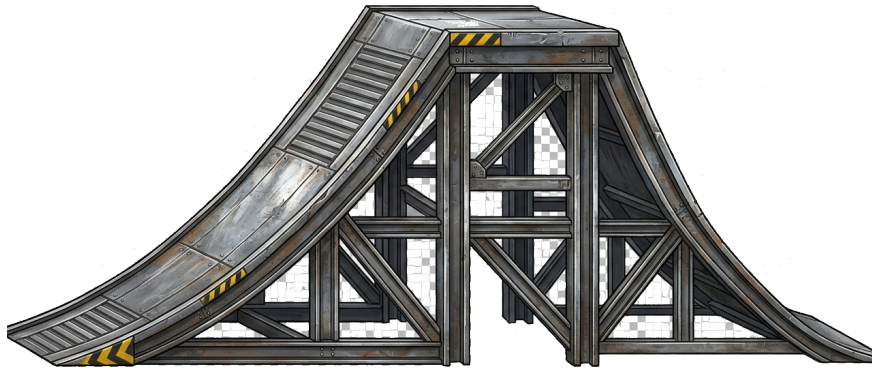


Figure 2: Level 1 in play. The HUD along the top shows the fuel bar, the coin count, the score and the route position (1 of 8). The bike rides on the generated road drawn in front of the background, with a row of coins ahead of it.

The two figures below show the artwork the levels are built from, taken from the project's own asset folders.



*Figure 3: One of the five Level 2 ramps. The riding surface used by the physics is measured from this picture's own alpha channel, so the bike climbs the shape that is drawn.*



*Figure 4: The tank used in Level 3. It rolls towards the bike and holds the road, either as part of a robber fight or as the army checkpoint that the GIVEPERMISSION code clears.*

## 5. GitHub Collaboration

### 5.1 Repository Management

The project was kept in a shared GitHub repository so that the three of us could work on it at the same time, keep a history of the changes, and share the source and the assets without passing files around by hand.

### 5.2 Contribution Workflow

1. The repository was created with the project structure and the initial iGraphics setup.
2. Each member cloned it and worked on their own copy.
3. Features were developed separately so that unfinished work did not break the others' levels.
4. Changes were committed with messages describing what was added or fixed.
5. Work was reviewed before being merged into the main branch.
6. The merged build was tested again before the next piece of work started.

### 5.3 Individual Contributions

| Member ID      | Name                         | Contribution                                                                                                                                                                                                                                                                                                                                                       |
|----------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 00725105101164 | Mujahidul Islam Ansary Niloy | Level 2 gameplay development, bike and obstacle implementation, ramp structure, game mechanics integration and overall level setup. Level 3 core gameplay and the 13-background progression; the Menu and Intro system and the bike / on-foot combat; enemy, shooting, ramp, Volt Speed and the major story events; the cheat codes and the Level Complete system. |
| 00725105101149 | Foysal Sami                  | Environment development, the coin system, obstacle placement, visual setup and gameplay testing adjustments. Bike movement and controls; the coin and fuel system; cars, tanks and obstacle placement; collision detection and the Game Over system; on-foot movement.                                                                                             |
| 00725105101152 | Parijat Sarker               | Background, road and asset integration; music and sound integration; the HUD, timer and subtitle implementation; gameplay transitions, testing and debugging. Player and gameplay support, level testing, and the adjustments needed to keep the game running smoothly.                                                                                            |

## 6. Challenges & Limitations

### 6.1 Challenges

| Challenge                                                                                                                           | Solution                                                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Most of the supplied artwork had the transparency chequerboard painted into the pixels, so it could not be drawn over a background. | The background colour is sampled around the border and flood filled away when the picture is loaded, and the picture is then cropped to the pixels that are actually visible.                        |
| Ramps and obstacles floated above the road, because the PNG files have large empty margins around the artwork.                      | Every ramp and obstacle is cropped to its visible pixels at load time, so the bottom of the picture is the bottom of the object and it stands on the road.                                           |
| The bike jumped so high off the bigger ramps that it left the top of the screen.                                                    | The launch is limited using the ballistic height formula, so the arc tops out just below the HUD while a faster approach still gives a higher jump.                                                  |
| After the direction change in Level 3 the backgrounds disappeared completely.                                                       | The drawing loop still stopped at the first panel that was off screen, which is the wrong test once the panels run the other way. The test was corrected and BG1 to BG5 were dropped from the world. |
| The obstacle after one of the Level 2 ramps could not be cleared at every approach speed.                                           | The jump was simulated at each speed and the gap between the ramp and its obstacle was set to a distance that works from any approach.                                                               |
| Fuel cans placed too close to an obstacle made the pickup and the jump fight each other.                                            | The fuel positions were moved into the clear stretches, at least 800 pixels away from any obstacle.                                                                                                  |
| Opening several MCI devices for music and sound effects made the audio unreliable.                                                  | Each track and effect is opened once under its own alias and then started and stopped by name, so a level's music is never restarted when the background changes.                                    |

### 6.2 Limitations

- Single player only. There is no second player and no multiplayer mode.
- Three levels. The game ends after Level 3.
- The window is fixed at  $1280 \times 720$  and cannot be resized, and there is no full-screen mode.
- There is no save or resume. Closing the game loses the current run, and progress is not carried between sessions.
- Loading takes about twelve seconds at start-up, because every level's artwork is decoded and uploaded before the window appears.
- A few optional sound effects are supported by the code but the audio files are not included, so those actions are silent.

## **7. Conclusion & Future Improvements**

### **7.1 Summary**

The project delivers a complete and playable 2D motorcycle game built in C++ with the iGraphics library. It has three levels with their own routes, artwork and challenges, a menu and level selection, coins and fuel, obstacle and enemy collision, an on-foot combat mode, cheat codes, dialogue, music and sound, and proper game over and level complete handling.

Building it taught us a good deal about how a game is actually put together: how a scrolling world and a camera relate to each other, why collision shapes have to be derived from the artwork rather than guessed, how to keep physics stable so a vehicle neither floats nor sinks, and how much of the work in a game is careful placement and testing rather than new code.

### **7.2 Future Enhancements**

- More levels, reusing the existing pattern of background panels and fixed object layouts.
- A career and upgrade system, so coins collected in one run can unlock faster bikes.
- More enemy types with different behaviour, instead of the two robber types.
- A save and resume system, and a stored best score for each level.
- Selectable difficulty, scaling fuel drain, enemy damage and obstacle spacing.
- A loading screen, so the start-up wait is shown to the player instead of a blank window.